

Longest String Chain (LeetCode 1048) — Understanding it as an LIS Dynamic Programming Problem

1. Introduction

When you first see Longest String Chain, it doesn't look anything like the Longest Increasing Subsequence (LIS) problem.

LIS deals with numbers.

Longest String Chain deals with strings.

Most beginners think this is a completely new DP problem.

It is not.

It is actually the exact same LIS DP pattern, where the only thing that changes is the condition that allows one element to extend another.

Once you understand this mapping, you'll notice that several DP problems are simply LIS with a different transition condition, such as:

- Longest Increasing Subsequence
- Largest Divisible Subset
- Longest String Chain
- Russian Doll Envelopes
- Best Team With No Conflicts

All of these share the same DP structure.

2. First Understand the LIS Pattern

Before understanding Longest String Chain, let's recall what LIS actually computes.

Suppose we have

`nums = [10,9,2,5,3,7,101,18]`

We define

`dp[i]`

as

Length of the Longest Increasing Subsequence ending at index i .

Notice one important word:

ending

This means we are forcing

$\text{nums}[i]$

to be the last element of our subsequence.

We are asking:

"If I must end my subsequence at $\text{nums}[i]$, what is the maximum length I can achieve?"

For example,

$\text{nums} = [1, 5, 2, 3]$

At

$i = 3$

$\text{nums}[i] = 3$

Possible increasing subsequences ending at 3 are

$[1, 3]$

Length = 2

and

$[1, 2, 3]$

Length = 3

So

$\text{dp}[3] = 3$

3. How is $\text{dp}[i]$ Computed in LIS?

Suppose

$\text{nums} = [1, 2, 4, 3]$

We're computing

`dp[3]`

Current number

`3`

We check every previous element.

`1`

`2`

`4`

Question:

Can 3 come after 1?

Yes.

Can 3 come after 2?

Yes.

Can 3 come after 4?

No.

Therefore,

```
dp[3]
=
max(
dp[0]+1,
dp[1]+1
)
```

This gives

`dp[3]=3`

General LIS Transition

For every

`j < i`

if

`nums[j] < nums[i]`

then

$$\begin{aligned} dp[i] &= \\ \max(dp[i], dp[j]+1) \end{aligned}$$

This is the entire LIS algorithm.

Notice something important.

The algorithm itself doesn't care about numbers.

It only needs one thing:

A rule that decides whether j can come before i.

In LIS that rule is

$$nums[j] < nums[i]$$

4. Converting LIS into Longest String Chain

Now replace numbers with strings.

Instead of

1
2
3
4

we have

"a"
"ab"
"abc"
"abcd"

Can

"ab"

come after

"a"

Yes.

Can

"abc"

come after

"ab"

Yes.

Can

"abcd"

come after

"abc"

Yes.

This looks exactly like an increasing sequence.

The only question becomes

How do we decide whether one string can come after another?

5. What is a Predecessor?

The problem defines a predecessor as

Word A

is predecessor of

Word B

if

Word B

can be obtained by inserting exactly one character into

Word A

Example

abc



abdc

Insert

d

between

b

and

c

Another example

abc



abcd

Insert

d

at the end.

Another

abc



xabc

Insert

x

at the beginning.

All are valid.

6. Replacing the LIS Condition

In LIS,

the extension rule is

$\text{nums}[j] < \text{nums}[i]$

meaning

`i`

can extend

`j`

In Longest String Chain,

the extension rule becomes

```
checkPossible(words[i], words[j])
```

meaning

`words[j]`

is predecessor of

`words[i]`

So

LIS

```
if(nums[j]<nums[i])
```

becomes

```
if(checkPossible(words[i], words[j]))
```

Everything else stays identical.

7. Why Do We Sort?

This is one of the most important observations.

Suppose the input is

```
["abcd", "a", "abc", "ab"]
```

If we process them directly,

`abcd`

comes before

`abc`

which is wrong.

We need every predecessor to appear first.

Therefore,

we sort by string length.

After sorting

a
ab
abc
abcd

Now

every possible predecessor automatically comes before its successor.

Exactly what LIS requires.

8. Why is the DP State Still the Same?

We define

dp[i]

as

Length of the longest string chain ending at words[i].

Notice this is exactly the same definition used in LIS.

Only

number

has become

string

Nothing else changes.

9. DP Transition

Suppose

Current word

abcd

Previous words

a

ab

abc

We ask

Can

abc

be predecessor?

Yes.

Therefore

abcd

can extend

abc

Exactly like LIS.

So

$$\begin{aligned} \text{dp}[\text{current}] \\ &= \\ \text{dp}[\text{abc}] + 1 \end{aligned}$$

If another predecessor exists,

we take the maximum.

Hence

$$\begin{aligned} \text{dp}[i] \\ &= \\ \max(\text{dp}[i], \text{dp}[j] + 1) \end{aligned}$$

Same recurrence.

10. Dry Run

Input

["a", "ab", "abc", "abcd"]

Initially

dp

1 1 1 1

i=1

Current

ab

Previous

a

Valid predecessor

dp[1]

=max(1, 1+1)

=2

DP

1 2 1 1

i=2

Current

abc

Previous

a

Not predecessor.

Skip.

Previous

ab

Valid predecessor.

dp[2]

=max(1, 2+1)

=3

DP

1 2 3 1

`i=3`

Current

`abcd`

Previous

`abc`

Valid predecessor.

`dp[3]`

`=max(1, 3+1)`

`=4`

Final

`1 2 3 4`

Answer

4

11. Why Doesn't the Algorithm Need to Know the Entire Chain?

Suppose

`abc`

already knows

`dp=3`

Then

`abcd`

doesn't need to know

`a→ab→abc`

It only needs

`dp[abc]`

which already stores

longest chain ending at abc

Therefore

`abcd`

simply extends it.

This is the essence of Dynamic Programming.

12. Mapping Between LIS and Longest String Chain

LIS	Longest String Chain
Element = Number	Element = String
dp[i] = LIS ending at i	dp[i] = Longest chain ending at i
nums[j] < nums[i]	checkPossible(words[i], words[j])
Extend subsequence	Extend string chain
Previous smaller number	Previous predecessor string
Take maximum	Take maximum

13. General Pattern

Once you recognize the pattern, you'll see many problems follow the same template:

```
Sort (if required)
for(i = 0 to n-1){
    dp[i] = 1;
    for(j = 0 to i-1){
        if(CanExtend(j, i)){
            dp[i] = max(dp[i], dp[j] + 1);
        }
    }
}
answer = max(dp);
```

The only thing that changes from one problem to another is the CanExtend(j, i) condition:

- LIS: $\text{nums}[j] < \text{nums}[i]$
- Largest Divisible Subset: $\text{nums}[i] \% \text{nums}[j] == 0$
- Longest String Chain: $\text{checkPossible}(\text{words}[i], \text{words}[j])$
- Russian Doll Envelopes: $\text{width}[j] < \text{width}[i] \ \&\& \ \text{height}[j] < \text{height}[i]$

The DP state, nested loops, transition, and answer extraction remain the same. This is why these problems are all considered variations of the LIS Dynamic Programming pattern.